

# Programování v C++ a C#

## Souhrn k maturitě

### Obsah

|                                           |   |
|-------------------------------------------|---|
| Programování v C++ a C#                   | 1 |
| Souhrn k maturitě                         | 1 |
| Základní řídicí struktury                 | 1 |
| Pojem algoritmus                          | 1 |
| Vlastnosti algoritmu                      | 1 |
| Způsoby zápisu algoritmu                  | 1 |
| Strukturované programování                | 1 |
| Sekvence                                  | 1 |
| Selekce (větvení)                         | 1 |
| Příkaz if                                 | 1 |
| Příkaz switch                             | 1 |
| Iterace (cyklus)                          | 1 |
| Cyklus while                              | 1 |
| Cyklus do-while                           | 1 |
| Cyklus for                                | 1 |
| Generování náhodných čísel                | 1 |
| C++                                       | 1 |
| C#                                        | 1 |
| Kolekce Controls v C#                     | 1 |
| Základní použití                          | 1 |
| Přístup k prvkům podle typu               | 1 |
| Přístup podle jména                       | 1 |
| Rekurzivní procházení                     | 1 |
| Jednoduché datové typy                    | 2 |
| Deklarace datových typů a proměnných      | 2 |
| C++                                       | 2 |
| C#                                        | 2 |
| Celočíselné datové typy                   | 2 |
| C++                                       | 2 |
| C#                                        | 2 |
| Reálné datové typy                        | 2 |
| C++                                       | 2 |
| C#                                        | 2 |
| Logický datový typ                        | 2 |
| C++                                       | 2 |
| C#                                        | 2 |
| Datový typ char                           | 2 |
| C++                                       | 2 |
| C#                                        | 2 |
| Třída Character v C#                      | 2 |
| Funkce C++                                | 2 |
| Pojem funkce                              | 2 |
| Definice funkce                           | 2 |
| Syntaxe                                   | 2 |
| Části funkce                              | 2 |
| Příklady                                  | 2 |
| Umístění definice funkce                  | 2 |
| Před funkcí main()                        | 2 |
| Po funkci main() s prototypem             | 2 |
| Hlavičkové soubory                        | 2 |
| Volání funkce                             | 2 |
| Formální parametry                        | 2 |
| Skutečné parametry (argumenty)            | 2 |
| Lokální a globální proměnné               | 2 |
| Lokální proměnné                          | 2 |
| Globální proměnné                         | 2 |
| Typy parametrů funkcí                     | 2 |
| Předávání hodnotou (by value)             | 2 |
| Předávání referencí (by reference)        | 2 |
| Předávání ukazatelem (by pointer)         | 2 |
| Konstantní parametry                      | 2 |
| Výchozí hodnoty parametrů                 | 2 |
| Přetěžování funkcí (Function Overloading) | 2 |
| Pravidla přetěžování                      | 2 |
| Příklady přetěžování                      | 2 |
| Praktický příklad – tisk různých typů     | 2 |
| Výhody přetěžování                        | 2 |
| Metody C#                                 | 3 |
| Pojem metoda                              | 3 |
| Deklarace a implementace metody           | 3 |
| Syntaxe                                   | 3 |
| Části metody                              | 3 |
| Příklady základních metod                 | 3 |
| Příkaz return                             | 3 |
| Volání metod                              | 3 |
| Statické metody                           | 3 |
| Instance metody (nestatické)              | 3 |
| Různé způsoby volání                      | 3 |
| Formální a skutečné parametry             | 3 |
| Formální parametry                        | 3 |
| Skutečné parametry (argumenty)            | 3 |
| Pojmenované argumenty                     | 3 |
| Volitelné parametry                       | 3 |
| Modifikátory parametrů                    | 3 |
| Předávání hodnotou (výchozí)              | 3 |
| Modifikátor ref                           | 3 |
| Modifikátor out                           | 3 |
| Modifikátor in (C# 7.2+)                  | 3 |
| Modifikátor params                        | 3 |
| Porovnání modifikátorů                    | 3 |
| Přetěžování metod (Method Overloading)    | 3 |
| Pravidla přetěžování                      | 3 |
| Příklady přetěžování                      | 3 |
| Přetěžování s modifikátory                | 3 |
| Praktický příklad                         | 3 |
| Výhody přetěžování                        | 3 |
| Datový typ pole                           | 4 |
| Jednorozměrné pole                        | 4 |
| Vlastnosti pole                           | 4 |
| Výhody polí                               | 4 |
| Nevýhody polí                             | 4 |
| Deklarace a inicializace pole             | 4 |
| C++ – pole                                | 4 |
| C# – pole                                 | 4 |
| Přístup k jednotlivým prvkům pole         | 4 |
| C++ – přístup k prvkům                    | 4 |
| C# – přístup k prvkům                     | 4 |
| Indexy od konce (C# 8.0+)                 | 4 |
| Praktické příklady                        | 4 |
| Pole jako parametr funkce                 | 4 |
| C++ – pole jako parametr                  | 4 |
| C# – pole jako parametr                   | 4 |
| Statické metody třídy Array (C#)          | 4 |
| Řazení pole                               | 4 |
| Vyhledávání v poli                        | 4 |
| Kopírování pole                           | 4 |
| Vyplnění pole                             | 4 |
| Další užitečné metody                     | 4 |
| Instanční metody pole (C#)                | 4 |
| Vlastnosti pole                           | 4 |
| Instance metody                           | 4 |
| LINQ metody pro pole                      | 4 |
| Příklady praktického použití              | 4 |

## Základní řídicí struktury

### Pojem algoritmus

**Algoritmus** je přesný návod (postup) jak řešit určitý problém. Jde o konečnou posloupnost jasně definovaných kroků, které vedou k vyřešení úlohy.

### Vlastnosti algoritmu

- Elementárnost** – algoritmus se skládá z konečného počtu jednoduchých (elementárních) kroků
- Konečnost** (finitnost) – algoritmus musí skončit v konečném počtu kroků
- Obecnost** – algoritmus řeší celou třídu úloh, ne pouze jeden konkrétní případ
- Determinovanost** – každý krok je jednoznačně definován, při opakovaném provedení se stejnými vstupními daty dostaneme stejné výsledky
- Rezultativnost** – algoritmus vždy dospěje k výsledku

### Způsoby zápisu algoritmu

- Slovní popis** – popis v přirozeném jazyce (čeština, angličtina)
- Vývojný diagram** – grafické znázornění pomocí symbolů a šipek
- Strukturogram** (Nassi-Shneidermanův diagram) – blokové schéma bez použití šipek
- Pseudokód** – zápis podobný programovacímu jazyku, ale zjednodušený a srozumitelný
- Programovací jazyk** – zápis v konkrétním programovacím jazyce (C++, C#, Python...)

### Strukturované programování

Programovací paradigma, které klade důraz na:

- Přehlednost a čitelnost kódu
- Použití vlastností a čítele tří základních řídicích struktur: **sekvence**, **selekce**, **iterace**
- Vyhýbání se příkazu GOTO
- Dekompozici problému na menší části (procedure, funkce)
- Jednodušší ladění a údržbu kódu

### Sekvence

**Sekvence** je základní řídicí struktura, kde se příkazy vykonávají postupně jeden za druhým v pořadí, jak jsou napsány.

cpp

```
1 // C++
2 int a = 5;
3 int b = 10;
4 int soucet = a + b;
5 cout << soucet << endl;
```

cs

```
1 // C#
2 int a = 5;
3 int b = 10;
4 int soucet = a + b;
5 Console.WriteLine(soucet);
```

### Selekce (větvení)

Umožňuje podmíněné vykonání příkazů na základě splnění podmínky.

#### Příkaz if

C++:

cpp

```
1 if (podminka) {
2     // kód, který se vykoná, když je podmínka splněna
3 }
4
5 if (podminka) {
6     // kód pro splněnou podmínku
7 } else {
8     // kód pro nesplněnou podmínku
9 }
10
11 if (podminka1) {
12     // kód 1
13 } else if (podminka2) {
14     // kód 2
15 } else {
16     // kód 3
17 }
```

C#:

cs

```
1 if (podminka) {
2     // kód, který se vykoná, když je podmínka splněna
3 }
4
5 if (podminka) {
6     // kód pro splněnou podmínku
7 } else {
8     // kód pro nesplněnou podmínku
9 }
10
11 if (podminka1) {
12     // kód 1
13 } else if (podminka2) {
14     // kód 2
15 } else {
16     // kód 3
17 }
```

#### Příkaz switch

Používá se pro vícenásobné větvení na základě hodnoty výrazu.

C++:

cpp

```
1 switch (promenna) {
2     case hodnota1:
3         // kód
4         break;
5     case hodnota2:
6         // kód
7         break;
8     default:
9         // kód pro ostatní případy
10        break;
11 }
```

C#:

cs

```
1 switch (promenna) {
2     case hodnota1:
3         // kód
4         break;
5     case hodnota2:
6         // kód
7         break;
8     default:
9         // kód pro ostatní případy
10        break;
11 }
```

### Iterace (cyklus)

Umožňuje opakované vykonávání bloku příkazů.

#### Cyklus while

Cyklus s podmínkou na začátku – testuje podmínku před každou iterací.

C++:

cpp

```
1 while (podminka) {
2     // kód, který se opakuje, dokud je podmínka splněna
3 }
```

C#:

cs

```
1 while (podminka) {
2     // kód, který se opakuje, dokud je podmínka splněna
3 }
```

#### Cyklus do-while

Cyklus s podmínkou na konci – tělo cyklu se provede vždy alespoň jednou.

C++:

cpp

```
1 do {
2     // kód, který se vykoná alespoň jednou
3 } while (podminka);
```

C#:

cs

```
1 do {
2     // kód, který se vykoná alespoň jednou
3 } while (podminka);
```

#### Cyklus for

Cyklus s řídicí proměnnou – používá se, když známe počet opakování.

C++:

cpp

```
1 for (int i = 0; i < 10; i++) {
2     // kód, který se opakuje 10x
3 }
```

C#:

cs

```
1 for (int i = 0; i < 10; i++) {
2     // kód, který se opakuje 10x
3 }
```

### Generování náhodných čísel

C++

cpp

```
1 #include <cstdlib> // pro rand() a srand()
2 #include <ctime>    // pro time()
3
4 // Inicializace generátoru (provést jednou na začátku programu)
5 srand(time(0));
6
7 // Generování náhodného čísla
8 int nahodneCislo = rand(); // 0 až RAND_MAX
9
10 // Náhodné číslo v rozsahu 0-99
11 int cislo = rand() % 100;
12
13 // Náhodné číslo v rozsahu 1-100
14 int cislo = rand() % 100 + 1;
15
16 // Náhodné číslo v rozsahu min-max
17 int cislo = rand() % (max - min + 1) + min;
```

C#

cs

```
1 // Vytvoření instance Random (provést jednou)
2 Random rnd = new Random();
3
4 // Generování náhodného čísla
5 int nahodneCislo = rnd.Next(); // nezáporné celé číslo
6
7 // Náhodné číslo 0-99
8 int cislo = rnd.Next(100);
9
10 // Náhodné číslo 1-100
11 int cislo = rnd.Next(1, 101); // horní mez je exkluzivní
12
13 // Náhodné desetinné číslo 0.0-1.0
14 double des = rnd.NextDouble();
```

### Kolekce Controls v C#

Kolekce **Controls** je vlastnost kontejnerových prvků (Form, Panel, GroupBox...) ve Windows Forms, která obsahuje všechny ovládací prvky (controls) umístěné v daném kontejneru.

#### Základní použití

cs

```
1 // Přístup ke všem ovládacím prvkům formuláře
2 foreach (Control ctrl in this.Controls) {
3     // Práce s jednotlivými prvky
4 }
5
6 // Přidání ovládacího prvku
7 Button btn = new Button();
8 btn.Text = "Nové tlačítko";
9 this.Controls.Add(btn);
10
11 // Odebrání ovládacího prvku
12 this.Controls.Remove(btn);
13
14 // Počet prvků v kolekci
15 int pocet = this.Controls.Count;
```

#### Přístup k prvkům podle typu

cs

```
1 // Najít všechny TextBoxy
2 foreach (Control ctrl in this.Controls) {
3     if (ctrl is TextBox) {
4         TextBox tb = (TextBox)ctrl;
5         tb.Clear(); // Vymazání obsahu
6     }
7 }
8
9 // Pomocí LINQ
10 var textBoxy = this.Controls.OfType<TextBox>();
11 foreach (TextBox tb in textBoxy) {
12     tb.Clear();
13 }
```

#### Přístup podle jména

cs

```
1 // Najít prvek podle jména
2 Control ctrl = this.Controls["textBox1"];
3 if (ctrl != null && ctrl is TextBox) {
4     TextBox tb = (TextBox)ctrl;
5     tb.Text = "Nový text";
6 }
7
8 // Nebo pomocí Find
9 Control[] controls = this.Controls.Find("textBox1", true);
10 if (controls.Length > 0) {
11     TextBox tb = (TextBox)controls[0];
12 }
```

#### Rekurzivní procházení

Pro práci s vnorenými ovládacími kontejnery (Panel v Panelu atd.):

cs

```
1 private void ProchazejVsechnyPrvky(Control kontejner) {
2     foreach (Control ctrl in kontejner.Controls) {
3         // Zpracování prvku
4         Console.WriteLine(ctrl.Name);
5
6         // Rekurzivní volání pro vnořené kontejnery
7         if (ctrl.HasChildren) {
8             ProchazejVsechnyPrvky(ctrl);
9         }
10    }
11 }
12
13 // Použití
14 ProchazejVsechnyPrvky(this);
```



## Jednoduché datové typy

### Deklarace datových typů a proměnných

Proměnná je pojmenované místo v paměti, kde se uchovává hodnota určitého datového typu.

Deklarace proměnné znamená sdělení překladači, jaký datový typ a název bude proměnná mít.

Inicializace proměnné znamená přiřazení počáteční hodnoty při deklaraci.

```
C++  
1 // Deklarace bez inicializace  
2 int cislo;  
3 double pi;  
4  
5 // Deklarace s inicializací  
6 int vek = 25;  
7 double pi = 3.14159;  
8  
9 // Více proměnných najednou  
10 int a, b, c;  
11 int x = 10, y = 20, z = 30;  
12  
13 // Konstanty (nelze měnit)  
14 const int MAX = 100;  
15 const double E = 2.71828;
```

```
C#  
1 // Deklarace bez inicializace  
2 int cislo;  
3 double pi;  
4  
5 // Deklarace s inicializací  
6 int vek = 25;  
7 double pi = 3.14159;  
8  
9 // Více proměnných najednou  
10 int a, b, c;  
11 int x = 10, y = 20, z = 30;  
12  
13 // Konstanty (nelze měnit)  
14 const int MAX = 100;  
15 const double E = 2.71828;  
16  
17 // Implicitně typované proměnné (var)  
18 var jmeno = "Jan"; // string  
19 var pocet = 42; // int
```

### Celočíselné datové typy

Slouží k ukládání celých čísel (bez desetinné čárky).

```
C++  


| Datový typ         | Velikost | Rozsah                                    | Popis                     |
|--------------------|----------|-------------------------------------------|---------------------------|
| short              | 2 B      | –32 768 až 32 767                         | Krátké celé číslo         |
| int                | 4 B      | –2 147 483 648 až 2 147 483 647           | Celé číslo                |
| long               | 4/8 B    | závislé na systému                        | Dlouhé celé číslo         |
| long long          | 8 B      | –9·10 <sup>18</sup> až 9·10 <sup>18</sup> | Velmi dlouhé celé číslo   |
| unsigned short     | 2 B      | 0 až 65 535                               | Neznaménkové krátké       |
| unsigned int       | 4 B      | 0 až 4 294 967 295                        | Neznaménkové celé         |
| unsigned long      | 4/8 B    | závislé na systému                        | Neznaménkové dlouhé       |
| unsigned long long | 8 B      | 0 až 1.8·10 <sup>19</sup>                 | Neznaménkové velmi dlouhé |


```

```
1 short maleCislo = 100;  
2 int cislo = 50000;  
3 long velkeCislo = 1000000L;  
4 long long obrovskeCislo = 9223372036854775807LL;  
5  
6 unsigned int nezaporneCislo = 4080000000U;
```

```
C#  


| Datový typ | Velikost | Rozsah                                    | Popis               |
|------------|----------|-------------------------------------------|---------------------|
| sbyte      | 1 B      | –128 až 127                               | Znaménkový byte     |
| byte       | 1 B      | 0 až 255                                  | Neznaménkový byte   |
| short      | 2 B      | –32 768 až 32 767                         | Krátké celé číslo   |
| ushort     | 2 B      | 0 až 65 535                               | Neznaménkové krátké |
| int        | 4 B      | –2 147 483 648 až 2 147 483 647           | Celé číslo          |
| uint       | 4 B      | 0 až 4 294 967 295                        | Neznaménkové celé   |
| long       | 8 B      | –9·10 <sup>18</sup> až 9·10 <sup>18</sup> | Dlouhé celé číslo   |
| ulong      | 8 B      | 0 až 1.8·10 <sup>19</sup>                 | Neznaménkové dlouhé |


```

```
1 sbyte maleZnamenkove = -50;  
2 byte maleByte = 200;  
3 short kratke = 10000;  
4 ushort kratkeNeznamenkove = 50000;  
5 int cislo = 1000000;  
6 uint neznamenkove = 3000000000;  
7 long velke = 900000000000000000L;  
8 ulong velkeNeznamenkove = 10000000000000000000UL;
```

### Reálné datové typy

Slouží k ukládání čísel s desetinnou čárkou (plovoucí řádová čárka).

```
C++  


| Datový typ  | Velikost | Rozsah                 | Přesnost  |
|-------------|----------|------------------------|-----------|
| float       | 4 B      | ±3.4·10 <sup>38</sup>  | 7 číslic  |
| double      | 8 B      | ±1.7·10 <sup>308</sup> | 15 číslic |
| long double | 8-16 B   | závislé na systému     | 19 číslic |


```

```
1 float pi = 3.14f; // suffix 'f' pro float  
2 double e = 2.71828; // implicitně double  
3 long double presne = 3.141592653589793238L;  
4  
5 // Vádecká notace  
6 double velke = 1.5e10; // 1.5 · 1010  
7 float male = 2.3e-5f; // 2.3 · 10-5
```

```
C#  


| Datový typ | Velikost | Rozsah                 | Přesnost     |
|------------|----------|------------------------|--------------|
| float      | 4 B      | ±3.4·10 <sup>38</sup>  | 7 číslic     |
| double     | 8 B      | ±1.7·10 <sup>308</sup> | 15-16 číslic |
| decimal    | 16 B     | ±7.9·10 <sup>28</sup>  | 28-29 číslic |


```

```
1 float pi = 3.14f; // suffix 'f' pro float  
2 double e = 2.71828; // OR (logický součet) -> true  
3 decimal cena = 19.99m; // suffix 'm' pro decimal  
4  
5 // Vádecká notace  
6 double velke = 1.5e10; // 1.5 · 1010  
7 float male = 2.3e-5f; // 2.3 · 10-5
```

**Poznámka:** Typ decimal v C# je vhodný pro finanční výpočty, kde je potřeba vysoká přesnost a správné zaokrouhlování.

### Logický datový typ

Slouží k ukládání pravdivostních hodnot (pravda/nepravda).

```
C++  
1 bool pravda = true;  
2 bool nepravda = false;  
3  
4 // Výsledky porovnání  
5 bool vetsi = (5 > 3); // true  
6 bool mensi = (10 < 5); // false  
7 bool rovno = (7 == 7); // true  
8  
9 // Logické operace  
10 bool a = true && false; // AND (logický součin) -> false  
11 bool b = true || false; // OR (logický součet) -> true  
12 bool c = !true; // NOT (negace) -> false  
13  
14 // V podmínkách  
15 if (pravda) {  
16     cout << "Podmínka je splněna" << endl;  
17 }
```

```
C#  
1 bool pravda = true;  
2 bool nepravda = false;  
3  
4 // Výsledky porovnání  
5 bool vetsi = (5 > 3); // true  
6 bool mensi = (10 < 5); // false  
7 bool rovno = (7 == 7); // true  
8  
9 // Logické operace  
10 bool a = true && false; // AND (logický součin) -> false  
11 bool b = true || false; // OR (logický součet) -> true  
12 bool c = !true; // NOT (negace) -> false  
13  
14 // V podmínkách  
15 if (pravda) {  
16     Console.WriteLine("Podmínka je splněna");  
17 }
```

### Datový typ char

Slouží k ukládání jednotlivých znaků.

```
C++  
1 char znak = 'A';  
2 char cislo = '5';  
3 char mezera = ' ';  
4 char novyRadek = '\n';  
5 char tabulator = '\\t';  
6  
7 // Escape sekvence  
8 char uvozovky = '\\';  
9 char zpetneKositko = '\\\\';  
10  
11 // ASCII hodnoty  
12 char a = 65; // 'A'  
13 int ascii = 'A'; // 65  
14  
15 // Práce se znaky  
16 char velke = 'A';  
17 char male = velke + 32; // 'a' (v ASCII tabulce)  
18  
19 // Funkce pro práci se znaky (v <ctype>)  
20 #include <ctype>  
21  
22 bool jePismeno = isalpha('A'); // true  
23 bool jeCislice = isdigit('5'); // true  
24 bool jeVelke = isupper('A'); // true  
25 bool jeMale = islower('a'); // true  
26  
27 char naMale = tolower('A'); // 'a'  
28 char naVelke = tolower('a'); // 'A'
```

```
C#  
1 char znak = 'A';  
2 char cislo = '5';  
3 char mezera = ' ';  
4 char novyRadek = '\n';  
5 char tabulator = '\\t';  
6  
7 // Escape sekvence  
8 char uvozovky = '\\';  
9 char zpetneKositko = '\\\\';  
10  
11 // Unicode hodnoty  
12 char a = (char)65; // 'A'  
13 int unicode = 'A'; // 65  
14  
15 // Lze použít i Unicode escape  
16 char srdce = '\\u2665'; // ♥
```

### Třída Charakter v C#

C# poskytuje statickou třídu Char (nebo Character jako alias) s užitečnými metodami:

```
1 // Testovací metody  
2 bool jePismeno = Char.IsLetter('A'); // true  
3 bool jeCislice = Char.IsDigit('5'); // true  
4 bool jeMezera = Char.IsWhiteSpace(' '); // true  
5 bool jeVelke = Char.IsUpper('A'); // true  
6 bool jeMale = Char.IsLower('a'); // true  
7 bool jePismenoCisCislo = Char.IsLetterOrDigit('A'); // true  
8  
9 // Konverzní metody  
10 char naMale = Char.ToLower('A'); // 'a'  
11 char naVelke = Char.ToUpper('a'); // 'A'  
12  
13 // Získání numerické hodnoty  
14 int hodnota = Char.GetNumericValue('5'); // 5.0  
15  
16 // Porovnání znaků  
17 int porovnaní = Char.Compare('A', 'B'); // -1 (A < B)  
18  
19 // Příklad použití  
20 char vstup = 'X';  
21 if (Char.IsUpper(vstup)) {  
22     Console.WriteLine($"vstup je velké písmeno");  
23     char male = Char.ToLower(vstup);  
24     Console.WriteLine($"Malé písmeno: {male}");  
25 }
```

## Funkce C++

### Pojem funkce

Funkce je pojmenovaný blok kódu, který provádí určitou úlohu. Funkce umožňují:

- **Znovupoužitelnost kódu** – jednou napsaný kód lze volat vícekrát
- **Modularitu** – rozdělení programu na menší, logické celky
- **Čitelnost** – přehlednější a srozumitelnější kód
- **Údržbu** – snadnější opravy a změny
- **Abstrakci** – skrytí implementačních detailů

### Definice funkce

Funkce v C++ má dvě části: **deklaraci** (prototyp) a **implementaci** (tělo funkce).

```
Syntaxe  
1 // Deklarace (prototyp) – říká překladači, že funkce existuje  
2 navratovytyp nazevFunkce(typParametru1 parametr1, typParametru2 parametr2);  
3  
4 // Implementace (definice) – obsahuje kód funkce  
5 navratovytyp nazevFunkce(typParametru1 parametr1, typParametru2 parametr2) {  
6     // tělo funkce  
7     return hodnota; // pokud návratový typ není void  
8 }
```

#### Části funkce

1. **Návratový typ** – typ hodnoty, kterou funkce vrátí (int, double, void...)
2. **Název funkce** – identifikátor funkce
3. **Seznam parametrů** – vstupy funkce (může být prázdný)
4. **Tělo funkce** – kód omezený složenými závorkami {}
5. **Příkaz return** – vrací hodnotu (kromě funkcí typu void)

```
cpp  
1 // Funkce bez parametrů a bez návratové hodnoty  
2 void pozdrav() {  
3     cout << "Ahoj!" << endl;  
4 }  
5  
6 // Funkce s parametry a návratovou hodnotou  
7 int soucet(int a, int b) {  
8     return a + b;  
9 }  
10  
11 // Funkce s více parametry  
12 double objem(double delka, double sirka, double vyska) {  
13     return delka * sirka * vyska;  
14 }  
15  
16 // Funkce vracějící boolean  
17 bool jeKladne(int cislo) {  
18     return cislo > 0;  
19 }
```

### Umístění definice funkce

```
cpp  
1 #include <iostream>  
2 using namespace std;  
3  
4 // Funkce definovaná před main() – není potřeba prototyp  
5 int soucet(int a, int b) {  
6     return a + b;  
7 }  
8  
9 int main() {  
10     int vysledek = soucet(5, 3);  
11     cout << vysledek << endl;  
12     return 0;  
13 }
```

### Po funkci main() s prototypem

```
cpp  
1 #include <iostream>  
2 using namespace std;  
3  
4 // Prototyp (deklarace) před main()  
5 int soucet(int a, int b);  
6  
7 int main() {  
8     int vysledek = soucet(5, 3);  
9     cout << vysledek << endl;  
10     return 0;  
11 }  
12  
13 // Implementace po main()  
14 int soucet(int a, int b) {  
15     return a + b;  
16 }
```

### Hlavičkové soubory

Pro větší projekty se funkce rozdělují do samostatných souborů:

**matematika.h** (hlavičkový soubor – deklarace):

```
cpp  
1 #ifndef MATHEMATIKA_H  
2 #define MATHEMATIKA_H  
3  
4 int soucet(int a, int b);  
5 int rozdil(int a, int b);  
6 double prumer(int a, int b);  
7  
8 #endif
```

**matematika.cpp** (zdrojový soubor – implementace):

```
cpp  
1 #include "matematika.h"  
2  
3 int soucet(int a, int b) {  
4     return a + b;  
5 }  
6  
7 int rozdil(int a, int b) {  
8     return a - b;  
9 }  
10  
11 double prumer(int a, int b) {  
12     return (a + b) / 2.0;  
13 }
```

**main.cpp** (hlavní program):

```
cpp  
1 #include <iostream>  
2 #include "matematika.h"  
3 using namespace std;  
4  
5 int main() {  
6     cout << soucet(10, 5) << endl; // 15  
7     cout << rozdil(10, 5) << endl; // 5  
8     cout << prumer(10, 5) << endl; // 7.5  
9     return 0;  
10 }
```

### Volání funkce

Volání funkce znamená provedení (spuštění) kódu, který je v těle funkce definován.

```
cpp  
1 int soucet(int a, int b) { // a, b jsou FORMÁLNÍ parametry  
2     return a + b;  
3 }  
4  
5 int main() {  
6     int x = 10, y = 20;  
7     int vysledek = soucet(x, y); // x, y jsou SKUTEČNÉ parametry  
8     // nebo přímo:  
9     int vysledek2 = soucet(5, 8); // 5, 8 jsou SKUTEČNÉ parametry  
10     return 0;  
11 }
```

### Formální parametry

- Parametry v definici funkce
- Specifikují typ a název vstupních hodnot
- Existují pouze během provádění funkce
- Funkční jako lokální proměnné

### Skutečné parametry (argumenty)

- Hodnoty předané při volání funkce
- Mohou být: konstanty, proměnné, výrazy
- Musí odpovídat typem formálních parametrů
- Počet a pořadí musí souhlasit

```
cpp  
1 double mocnina(double zaklad, int exponent) {  
2     double vysledek = 1;  
3     for (int i = 0; i < exponent; i++) {  
4         vysledek *= zaklad;  
5     }  
6     return vysledek;  
7 }  
8  
9 int main() {  
10     // Různé způsoby volání  
11     double a = mocnina(2.0, 3); // s proměnnými  
12     double b = mocnina(5, 2); // s konstantami  
13     double c = mocnina(1.5, 2 + 2); // s výrazem  
14     return 0;  
15 }
```

### Lokální a globální proměnné

#### Lokální proměnné

- Deklarované **uvnitř** funkce nebo bloku
- Existují pouze během provádění funkce
- Každé volání funkce vytváří novou instanci
- Po ukončení funkce jsou zničeny
- Mají přednost před globálními proměnnými se stejným názvem

```
cpp  
1 void funkce() {  
2     int lokalniPromenna = 10; // lokální proměnná  
3     // použitelná jen v této funkci  
4 }  
5  
6 int main() {  
7     int x = 5; // lokální v main()  
8     funkce();  
9     // zde nelze použít lokalniPromenna ani x z funkce()  
10     return 0;  
11 }
```

#### Globální proměnné

- Deklarované **mimo** všechny funkce
- Dostupné ve všech funkcích v programu
- Existují po celou dobu běhu programu
- Uchovávají hodnotu mezi voláními funkcí
- **Nedoporučuje se nadužívat** – horší čitelnost a údržba

```
cpp  
1 #include <iostream>  
2 using namespace std;  
3  
4 int globalniPromenna = 100; // globální proměnná  
5  
6 void funkce1() {  
7     cout << globalniPromenna << endl; // přístup k globální  
8     globalniPromenna = 200; // změna globální  
9 }  
10  
11 void funkce2() {  
12     int globalniPromenna = 50; // lokální – zastíní globální  
13     cout << globalniPromenna << endl; // vypíše 50  
14     cout << :globalniPromenna << endl; // :: přístup ke globální  
15 }  
16  
17 int main() {  
18     funkce1(); // vypíše 100, změní na 200  
19     funkce1(); // vypíše 200  
20     funkce2(); // vypíše 50, pak 200  
21     return 0;  
22 }
```

### Typy parametrů funkcí

#### Předávání hodnotou (by value)

- Vytváří se **kopie** skutečného parametru
- Změny uvnitř funkce neovlivní původní proměnnou
- Vhodné pro malé datové typy

```
cpp  
1 void zmena(int x) {  
2     x = 100; // změní pouze lokální kopii  
3 }  
4  
5 int main() {  
6     int a = 5;  
7     zmena(a);  
8     cout << a << endl; // vypíše 5 (původní hodnota)  
9     return 0;  
10 }
```

#### Předávání referencí (by reference)

- Používá se symbol &
- Nepředává se kopie, ale **odkaz** na originální proměnnou
- Změny uvnitř funkce **ovlivní** původní proměnnou
- Efektivnější pro velké datové struktury
- Umožňuje vracet více hodnot

```
cpp  
1 void zmena(int &x) { // ukazuje referenci  
2     x = 100; // změní původní proměnnou  
3 }  
4  
5 void prohod(int &a, int &b) {  
6     int temp = a;  
7     a = b;  
8     b = temp;  
9 }  
10  
11 int main() {  
12     int a = 5;  
13     zmena(a);  
14     cout << a << endl; // vypíše 100 (změněná hodnota)  
15  
16     int x = 10, y = 20;  
17     prohod(x, y);  
18     cout << x << ", " << y << endl; // vypíše 20, 10  
19     return 0;  
20 }
```

#### Předávání ukazatelem (by pointer)

- Používá se symbol \* pro typ a & pro získání adresy
- Předává se **adresa** proměnné v paměti
- Funkce získá s adresou pomocí operátoru dereference \*
- Starší způsob, reference jsou modernější

```
cpp  
1 void zmena(int *x) { // ukazatel na int  
2     *x = 100; // změna hodnoty na adrese  
3 }  
4  
5 int main() {  
6     int a = 5;  
7     zmena(&a); // předání adresy proměnné a  
8     cout << a << endl; // vypíše 100  
9     return 0;  
10 }
```

### Konstantní parametry

- Používá se klíčové slovo const
- Zabrání změně hodnoty parametru
- Užitečné pro velké objekty předávané referencí

```
cpp  
1 void vycis(const int &x) { // konstantní reference  
2     cout << x << endl;  
3     // x = 10; // CHYBA – nelze měnit const parametr  
4 }  
5  
6 double obvod(const double pi, const double r) {  
7     return 2 * pi * r;  
8     // pi = 3; // CHYBA – nelze měnit  
9 }
```

### Výchozí hodnoty parametrů

- Parametry mohou mít výchozí hodnoty
- Výchozí hodnoty se zadávají v deklaraci (prototypu)
- Parametry s výchozí hodnotou musí být na konci seznamu

```
cpp  
1 // V prototypu nebo definici  
2 void tiskni(string text, int pocet = 1) {  
3     for (int i = 0; i < pocet; i++) {  
4         cout << text << endl;  
5     }  
6 }  
7  
8 int main() {  
9     tiskni("Ahoj"); // použije výchozí pocet = 1  
10    tiskni("Ahoj", 3); // použije pocet = 3  
11    return 0;  
12 }
```

### Přetěžování funkcí (Function Overloading)

Přetěžování funkcí umožňuje definovat více funkcí se stejným názvem, ale s různými parametry.

#### Pravidla přetěžování

Funkce se liší:

1. **Početem parametrů**
2. **Typy parametrů**
3. **Pořadím typů parametrů**

Funkce se **neliší** pouze návratovým typem!

```
cpp  
1 // Různý počet parametrů  
2 int soucet(int a, int b) {  
3     return a + b;  
4 }  
5  
6 int soucet(int a, int b, int c) {  
7     return a + b + c;  
8 }  
9  
10 // Různé typy parametrů  
11 double soucet(double a, double b) {  
12     return a + b;  
13 }  
14  
15 string soucet(string a, string b) {  
16     return a + b; // spojení řetězců  
17 }  
18  
19 // Použití  
20 int main() {  
21     cout << soucet(5, 3) << endl; // volá int, int  
22     cout << soucet(5, 3, 2) << endl; // volá int, int, int  
23     cout << soucet(2.5, 3.7) << endl; // volá double, double  
24     return 0; // volá string, string  
25 }
```

### Praktický příklad – tisk různých typů

```
cpp  
1 #include <iostream>  
2 using namespace std;  
3  
4 void vypis(int x) {  
5     cout << "Celé číslo: " << x << endl;  
6 }  
7  
8 void vypis(double x) {  
9     cout << "Reálné číslo: " << x << endl;  
10 }  
11  
12 void vypis(string x) {  
13     cout << "Text: " << x << endl;  
14 }  
15  
16 void vypis(int x, int y) {  
17     cout << "Dva integrity: " << x << ", " << y << endl;  
18 }  
19  
20 int main() {  
21     vypis(42); // volá vypis(int)  
22     vypis(3.14); // volá vypis(double)  
23     vypis("Hello"); // volá vypis(string)  
24     vypis(10, 20); // volá vypis(int, int)  
25     return 0;  
26 }
```

### Výhody přetěžování

1. Jednoduché rozhraní – stejný název pro podobné operace
2. Čitelnější kód – není nutné vymýšlet různé názvy
3. Intuitivní použití – programátor nemusí pamatovat různé názvy
4. Polymorfismus – základ objektově orientovaného programování



# Metody C#

## Pojem metoda

**Metoda** je pojmenovaný blok kódu, který provádí specifickou úlohu. V C# jsou metody vždy součástí třídy nebo struktury.

Metody v C# poskytují:

- **Znovupoužitelnost** – stejný kód lze volat opakovaně
- **Organizaci** – logické seskupení funkcionality
- **Zapouzdření** – skrytí implementačních detailů
- **Abstrakci** – zjednodušení složitých operací
- **Modularitu** – rozdělení programu na menší části

Rozdíl mezi metodou a funkcí:

- V C++ mluvíme o **funkcích**
- V C# mluvíme o **metodách** (jsou vždy součástí třídy)
- Princip je stejný, terminologie se liší

## Deklarace a implementace metody

### Syntaxe

cs

```
1 // Základní struktura metody
2 modifikátorPřístupu návratovýTyp NavezMetody(typParametru1 paramet1, typParametru2
paramet2) {
3     // tělo metody
4     return hodnota; // pokud návratový typ není void
5 }
```

### Části metody

1. **Modifikátor přístupu** – public, private, protected, internal
2. **Návratový typ** – typ hodnoty, kterou metoda vrátí (int, string, void...)
3. **Název metody** – identifikátor metody (PascalCase v C#)
4. **Seznam parametrů** – vstupy metody (může být prázdný)
5. **Tělo metody** – kód ohraničený složenými závorkami
6. **Příkaz return** – vrací hodnotu volajícímu kódu

### Příklady základních metod

cs

```
1 class Program {
2     // Metoda bez parametrů a bez návratové hodnoty
3     static void Pozdrav() {
4         Console.WriteLine("Ahoj!");
5     }
6
7     // Metoda s parametry a návratovou hodnotou
8     static int Soucet(int a, int b) {
9         return a + b;
10    }
11
12    // Metoda s více parametry
13    static double Objem(double delka, double sirka, double vyska) {
14        return delka * sirka * vyska;
15    }
16
17    // Metoda vracející boolean
18    static bool JeKladne(int cislo) { // AH0J
19        return cislo > 0;
20    }
21
22    // Metoda vracející string
23    static string ZiskejPozdrav(string jmeno) {
24        return $"Ahoj, {jmeno}!";
25    }
26 }
```

### Příkaz return

Příkaz return má dva účely:

1. **Ukončí vykonávání metody** – žádný další kód po return se neprovede
2. **Vrátí hodnotu** – pokud metoda není typu void

cs

```
1 static int AbsolutniHodnota(int cislo) {
2     if (cislo < 0) {
3         return -cislo; // ukončí metodu zde
4     }
5     return cislo; // jinak vrátí původní hodnotu
6 }
7
8 static void VypisHodnotu(int x) {
9     if (x < 0) {
10        Console.WriteLine("Záporné číslo");
11        return; // předčasné ukončení (void metoda)
12    }
13    Console.WriteLine($"Kladné číslo: {x}");
14 }
```

## Volání metod

### Statické metody

Statické metody se volají přes název třídy nebo přímo (pokud jsou ve stejné třídě):

cs

```
1 class Matematika {
2     static int Soucet(int a, int b) {
3         return a + b;
4     }
5
6     static void Main() {
7         // Volání metody ve stejné třídě
8         int vysledek = Soucet(5, 3);
9
10        // Volání metody z jiné třídy
11        int vysledek2 = Matematika.Soucet(10, 20);
12    }
13 }
```

### Instance metody (nestatické)

Nestatické metody vyžadují instanci třídy:

cs

```
1 class Kalkulacka {
2     public int Soucet(int a, int b) {
3         return a + b;
4     }
5 }
6
7 class Program {
8     static void Main() {
9         Kalkulacka calc = new Kalkulacka();
10        int vysledek = calc.Soucet(5, 3); // volání přes instanci
11    }
12 }
```

### Různé způsoby volání

cs

```
1 class Demo {
2     static int Mocnina(int zaklad, int exponent) {
3         int vysledek = 1;
4         for (int i = 0; i < exponent; i++) {
5             vysledek *= zaklad;
6         }
7         return vysledek;
8     }
9
10    static void Main() {
11        // S proměnnými
12        int x = 2, y = 3;
13        int a = Mocnina(x, y);
14
15        // S konstantami
16        int b = Mocnina(5, 2);
17
18        // S výrazem
19        int c = Mocnina(3, 1 + 1);
20
21        // Vnořené volání
22        int d = Mocnina(Mocnina(2, 2), 2); // (2^2)^2 = 16
23
24        // Výsledek jako argument
25        Console.WriteLine(Mocnina(2, 5));
26    }
27 }
```

## Formální a skutečné parametry

### Formální parametry

- Parametry v **definici** metody
- Definují typ a název vstupních hodnot
- Fungují jako lokální proměnné uvnitř metody
- Existují pouze během provádění metody

### Skutečné parametry (argumenty)

- Hodnoty předané při **volání** metody
- Mohou být: literály, proměnné, výrazy
- Musí odpovídat typem a počtem formálních parametrům
- Pořadí musí souhlasit (nebo použít pojmenované argumenty)

cs

```
1 static double VypocetCeny(double cena, double dph, double sleva) {
2     // cena, dph, sleva jsou FORMALNI parametry
3     return cena * (1 + dph) * (1 - sleva);
4 }
5
6 static void Main() {
7     double zakladniCena = 1000;
8     double sazba = 0.21;
9     double slevaProcent = 0.1;
10
11    // zakladniCena, sazba, slevaProcent jsou SKUTEČNÉ parametry
12    double konecnaCena = VypocetCeny(zakladniCena, sazba, slevaProcent);
13
14    // Také lze použít literály jako skutečné parametry
15    double cena2 = VypocetCeny(500, 0.21, 0.05);
16 }
```

### Pojmenované argumenty

C# umožňuje předávat argumenty podle názvu:

cs

```
1 static void VytvorUzivatele(string jmeno, int vek, string mesto) {
2     Console.WriteLine($"{{jmeno}}, {{vek}} let, {{mesto}}");
3 }
4
5 static void Main() {
6     // Standardní pořadí
7     VytvorUzivatele("Jan", 25, "Praha");
8
9     // Pojmenované argumenty (libovolné pořadí)
10    VytvorUzivatele(mesto: "Brno", jmeno: "Petra", vek: 30);
11
12    // Kombinace
13    VytvorUzivatele("Karel", vek: 28, mesto: "Ostrava");
14 }
```

### Volitelné parametry

Parametry mohou mít výchozí hodnoty:

cs

```
1 static void Pozdrav(string jmeno, string zprava = "Ahoj") {
2     Console.WriteLine($"{{zprava}}, {{jmeno}}!");
3 }
4
5 static void Main() {
6     Pozdrav("Jan"); // použije výchozí "Ahoj"
7     Pozdrav("Petra", "Dobrý den"); // použije "Dobrý den"
8 }
```

## Modifikátory parametrů

C# nabízí různé způsoby předávání parametrů pomocí modifikátorů.

### Předávání hodnotou (výchozí)

- Standardní způsob bez modifikátoru
- Vytvoří se **kopie** hodnoty
- Změny uvnitř metody **neovlivní** původní proměnnou
- Vhodné pro hodnotové typy (int, double, bool...)

cs

```
1 static void Zmena(int x) {
2     x = 100; // změní pouze lokální kopii
3     Console.WriteLine($"Uvnitř metody: {{x}}");
4 }
5
6 static void Main() {
7     int a = 5;
8     Zmena(a);
9     Console.WriteLine($"Po volání: {{a}}"); // stále 5
10 }
```

### Modifikátor ref

- Předává **referenci** na původní proměnnou
- Změny uvnitř metody **ovlivní** původní proměnnou
- Proměnná **musí být inicializována** před voláním
- Při volání musí být použito klíčové slovo **ref**

cs

```
1 static void Zmena(ref int x) {
2     x = 100; // změní původní proměnnou
3 }
4
5 static void Prohod(ref int a, ref int b) {
6     int temp = a;
7     a = b;
8     b = temp;
9 }
10
11 static void Main() {
12     int cislo = 5;
13     Zmena(ref cislo);
14     Console.WriteLine(cislo); // vypíše 100
15
16     int x = 10, y = 20;
17     Prohod(ref x, ref y);
18     Console.WriteLine($"{{x}}, {{y}}"); // vypíše 20, 10
19 }
```

### Modifikátor out

- Podobný jako **ref**, ale proměnná  **nemusí být inicializována**
- Metoda **musí přiřadit** hodnotu parametru před ukončením
- Vhodné pro vrácení více hodnot z metody
- Při volání musí být použito klíčové slovo **out**

cs

```
1 static bool TryParse(string text, out int vysledek) {
2     try {
3         vysledek = int.Parse(text);
4         return true;
5     } catch {
6         vysledek = 0;
7         return false;
8     }
9 }
10
11 static void VratViceHodnot(int x, out int dvojnásobek, out int trojnásobek) {
12     dvojnásobek = x * 2;
13     trojnásobek = x * 3;
14 }
15
16 static void Main() {
17     // out - nemusí být inicializováno
18     int hodnota;
19     if (TryParse("123", out hodnota)) {
20         Console.WriteLine($"Převod OK: {{hodnota}}");
21     }
22
23     // C# 7.0+ inline deklarace
24     if (TryParse("456", out int cislo)) {
25         Console.WriteLine($"Převod OK: {{cislo}}");
26     }
27
28     // Vracení více hodnot
29     VratViceHodnot(5, out int dva, out int tri);
30     Console.WriteLine($"2x: {{dva}}, 3x: {{tri}}"); // 10, 15
31 }
```

### Modifikátor in (C# 7.2+)

- Předává referenci, ale **pouze pro čtení**
- Optimalizace pro velké struktury
- Zabrání změně hodnoty uvnitř metody
- Volitelné použití klíčového slova při volání

cs

```
1 static double VypocetVzdalenosti(in Vector3 bod1, in Vector3 bod2) {
2     // bod1 a bod2 nelze měnit
3     double dx = bod2.X - bod1.X;
4     double dy = bod2.Y - bod1.Y;
5     double dz = bod2.Z - bod1.Z;
6     return Math.Sqrt(dx * dx + dy * dy + dz * dz);
7 }
```

### Modifikátor params

- Umožňuje předat **proměnný počet** argumentů
- Argumenty jsou předány jako pole
- Musí být poslední parametr v seznamu
- Lze volat bez argumentů (prázdné pole)

cs

```
1 static int Soucet(params int[] cisla) {
2     int vysledek = 0;
3     foreach (int cislo in cisla) {
4         vysledek += cislo;
5     }
6     return vysledek;
7 }
8
9 static void Main() {
10    Console.WriteLine(Soucet()); // 0
11    Console.WriteLine(Soucet(5)); // 5
12    Console.WriteLine(Soucet(1, 2, 3)); // 6
13    Console.WriteLine(Soucet(10, 20, 30, 40, 50)); // 150
14
15    // Nebo předat pole přímo
16    int[] pole = { 1, 2, 3, 4 };
17    Console.WriteLine(Soucet(pole)); // 10
18 }
```

## Porovnání modifikátorů

| Modifikátor | Inicializace před | Přiřazení v metodě | Může měnit       |
|-------------|-------------------|--------------------|------------------|
| (žádný)     | Ne                | Ne                 | Ne (kopie)       |
| ref         | Ano               | Ne                 | Ano              |
| out         | Ne                | Ano (povinné)      | Ano              |
| in          | Ano               | Ne                 | Ne (readonly)    |
| params      | Ne                | Ne                 | Ano (prvky pole) |

## Přetěžování metod (Method Overloading)

**Přetěžování** metod umožňuje definovat více metod se **stejným názvem**, ale s **různými parametry**.

### Pravidla přetěžování

Metody se mohou lišit:

1. **Počet parametrů**
2. **Typy parametrů**
3. **Pořadím typů parametrů**
4. **Modifikátory parametrů** (ref, out, in)

Metody se **neliší** pouze:

- Návratovým typem
- Názvy parametrů
- Modifikátory přístupu

### Příklady přetěžování

cs

```
1 class Kalkulacka {
2     // Různý počet parametrů
3     public int Soucet(int a, int b) {
4         return a + b;
5     }
6
7     public int Soucet(int a, int b, int c) {
8         return a + b + c;
9     }
10
11    public int Soucet(int a, int b, int c, int d) {
12        return a + b + c + d;
13    }
14
15    // Různé typy parametrů
16    public double Soucet(double a, double b) {
17        return a + b; // spojení řetězců
18    }
19
20    public string Soucet(string a, string b) {
21        return a + b;
22    }
23
24    // Různé pořadí typů
25    public void Vypis(int cislo, string text) {
26        Console.WriteLine($"Číslo: {{cislo}}, Text: {{text}}");
27    }
28
29    public void Vypis(string text, int cislo) {
30        Console.WriteLine($"Text: {{text}}, Číslo: {{cislo}}");
31    }
32 }
```

### Přetěžování s modifikátory

cs

```
1 class Demo {
2     // Standardní předání hodnotou
3     public void Metoda(int x) {
4         Console.WriteLine("Předání hodnotou");
5     }
6
7     // Přetížení s ref
8     public void Metoda(ref int x) {
9         Console.WriteLine("Předání referencí (ref)");
10    }
11
12    // Přetížení s out
13    public void Metoda(out int x) {
14        x = 10;
15        Console.WriteLine("Předání výstupem (out)");
16    }
17 }
```

### Praktický příklad

cs

```
1 class FormatovaniTextu {
2     // Základní metoda
3     public string Formatuj(string text) {
4         return text.ToUpper();
5     }
6
7     // S délkou
8     public string Formatuj(string text, int maxDelka) {
9         string vysledek = text.ToUpper();
10        if (vysledek.Length > maxDelka) {
11            vysledek = vysledek.Substring(0, maxDelka) + "...";
12        }
13        return vysledek;
14    }
15
16    // S délkou a prefixem
17    public string Formatuj(string text, int maxDelka, string prefix) {
18        string vysledek = text.ToUpper();
19        if (vysledek.Length > maxDelka) {
20            vysledek = vysledek.Substring(0, maxDelka) + "...";
21        }
22        return prefix + vysledek;
23    }
24
25    static void Main() {
26        var fmt = new FormatovaniTextu();
27
28        Console.WriteLine(fmt.Formatuj("ahoj")); // AH0J
29        Console.WriteLine(fmt.Formatuj("ahoj světe", 6)); // AH0J S...
30        Console.WriteLine(fmt.Formatuj("ahoj", 10, ">>> ")); // >>> AH0J
31    }
32 }
```

### Výhody přetěžování

1. **Flexibilita** – jedna metoda pro různé situace
2. **Čitelnost** – není nutné pamatovat různé názvy
3. **Intuitivnost** – programátor může volat stejnou metodu různými způsoby
4. **Konzistence** – jednotné ohraničení pro podobné operace
5. **Polymorfismus** – základ OOP v C#



## Datový typ pole

### Jednorozměrné pole

**Pole** (array) je datová struktura, která uchovává **pevný počet** prvků **stejného datového typu** uspořádaných v **lineární posloupnosti**.

#### Vlastnosti pole

- Pevná velikost** – po vytvoření nelze změnit
- Homogenní** – všechny prvky mají stejný datový typ
- Indexované** – přístup k prvkům pomocí indexu (číselné pozice)
- Nultý index** – první prvek má index 0
- Souvislá paměť** – prvky jsou uloženy vedle sebe v paměti

#### Výhody polí

- Rychlý přístup k prvkům (konstantní čas *O*(1))
- Effektivní využití paměti
- Jednoduché procházení prvků
- Vhodné pro ukládání sekvencí dat

#### Nevýhody polí

- Pevná velikost – nelze dynamicky měnit
- Vkládání a mazání prvků je neefektivní
- Plytvání pamětí, pokud není celé pole využito

### Deklarace a inicializace pole

#### C++ – pole

##### Deklarace pole:

cpp

```
1 typPrvku nazevPole[velikost];
```

##### Příklady deklarace a inicializace:

cpp

```
1 // Deklarace bez inicializace (prvky mají náhodné hodnoty)
2 int cisla[5];
3
4 // Deklarace s inicializací – explicitní velikost
5 int cisla[5] = {10, 20, 30, 40, 50};
6
7 // Deklarace s inicializací – implicitní velikost
8 int cisla[] = {10, 20, 30, 40, 50}; // velikost 5
9
10 // Částečná inicializace (zbytek vyplněn nulami)
11 int cisla[5] = {10, 20}; // {10, 20, 0, 0, 0}
12
13 // Inicializace všech prvků na nulu
14 int cisla[5] = {}; // {0, 0, 0, 0, 0}
15
16 // Pole různých typů
17 double teploty[7];
18 char znaky[10];
19 bool stavy[3] = {true, false, true};
20 string jmena[4] = {"Jan", "Petra", "Karel", "Anna"};
```

#### Dynamické pole (C++):

cpp

```
1 // Dynamická alokace pomocí new
2 int velikost = 10;
3 int* pole = new int[velikost];
4
5 // Použití pole
6 pole[0] = 5;
7 pole[1] = 10;
8
9 // Uvolnění paměti
10 delete[] pole;
```

#### Zjištění velikosti pole:

cpp

```
1 int cisla[] = {1, 2, 3, 4, 5};
2 int velikost = sizeof(cisla) / sizeof(cisla[0]); // 5
```

#### C# – pole

##### Deklarace pole:

cs

```
1 typPrvku[] nazevPole;
```

##### Příklady deklarace a inicializace:

cs

```
1 // Deklarace bez inicializace
2 int[] cisla;
3
4 // Vytvoření pole s danou velikostí (prvky inicializovány na 0)
5 int[] cisla = new int[5]; // {0, 0, 0, 0, 0}
6
7 // Deklarace a inicializace s hodnotami
8 int[] cisla = new int[] {10, 20, 30, 40, 50};
9
10 // Zkrácený zápis (typ lze vynechat)
11 int[] cisla = {10, 20, 30, 40, 50};
12
13 // Pole různých typů
14 double[] teploty = new double[7];
15 char[] znaky = {'A', 'B', 'C'};
16 bool[] stavy = {true, false, true};
17 string[] jmena = {"Jan", "Petra", "Karel", "Anna"};
18
19 // Implicitně typované pole
20 var cisla = new[] {1, 2, 3, 4, 5}; // typ int[]
```

#### Zjištění velikosti pole:

cs

```
1 int[] cisla = {1, 2, 3, 4, 5};
2 int velikost = cisla.Length; // 5
```

### Přístup k jednotlivým prvkům pole

Prvky pole jsou indexované od 0. K prvkům přistupujeme pomocí hranatých závorek [].

#### C++ – přístup k prvkům

cpp

```
1 int cisla[5] = {10, 20, 30, 40, 50};
2
3 // Čtení hodnot
4 int prvni = cisla[0]; // 10
5 int druhy = cisla[1]; // 20
6 int posledni = cisla[4]; // 50
7
8 // Zápis hodnot
9 cisla[0] = 100;
10 cisla[2] = 300;
11
12 // Výpis všech prvků pomocí cyklu for
13 for (int i = 0; i < 5; i++) {
14     cout << cisla[i] << " ";
15 }
16
17 // Range-based for loop (C++11)
18 for (int cislo : cisla) {
19     cout << cislo << " ";
20 }
21
22 // Pozor na přístup mimo rozsah (undefined behavior)
23 int x = cisla[10]; // NEBEZPEČNÉ!
```

#### C# – přístup k prvkům

cs

```
1 int[] cisla = {10, 20, 30, 40, 50};
2
3 // Čtení hodnot
4 int prvni = cisla[0]; // 10
5 int druhy = cisla[1]; // 20
6 int posledni = cisla[4]; // 50
7
8 // Zápis hodnot
9 cisla[0] = 100;
10 cisla[2] = 300;
11
12 // Výpis všech prvků pomocí cyklu for
13 for (int i = 0; i < cisla.Length; i++) {
14     Console.WriteLine(cisla[i] + " ");
15 }
16
17 // Foreach cyklus
18 foreach (int cislo in cisla) {
19     Console.WriteLine(cislo + " ");
20 }
21
22 // Přístup mimo rozsah vyvolá výjimku
23 int x = cisla[10]; // IndexOutOfRangeException
```

#### Indexy od konce (C# 8.0+)

cs

```
1 int[] cisla = {10, 20, 30, 40, 50};
2
3 // Index od konce pomocí operátoru ^
4 int posledni = cisla[^1]; // 50 (poslední prvek)
5 int predposledni = cisla[^2]; // 40 (předposlední prvek)
6
7 // Rozsahy (ranges)
8 int[] cast = cisla[1..4]; // {20, 30, 40}
9 int[] prvniTri = cisla[..3]; // {10, 20, 30}
10 int[] posledniDva = cisla[^2..]; // {40, 50}
```

### Praktické příklady

#### Součet prvků pole:

cpp

```
1 // C++
2 int soucet = 0;
3 for (int i = 0; i < velikost; i++) {
4     soucet += cisla[i];
5 }
6
7 // C#
8 int soucet = 0;
9 foreach (int cislo in cisla) {
10     soucet += cislo;
11 }
12
```

cs

```
1 // C#
2 int max = cisla[0];
3 for (int i = 1; i < cisla.Length; i++) {
4     if (cisla[i] > max) {
5         max = cisla[i];
6     }
7 }
8
9 // C++
10 int max = cisla[0];
11 for (int i = 1; i < velikost; i++) {
12     if (cisla[i] > max) {
13         max = cisla[i];
14     }
15 }
16
```

#### Nalezení maxima:

cpp

```
1 // C++
2 int max = cisla[0];
3 for (int i = 1; i < velikost; i++) {
4     if (cisla[i] > max) {
5         max = cisla[i];
6     }
7 }
8
9 // C#
10 int max = cisla[0];
11 for (int i = 1; i < cisla.Length; i++) {
12     if (cisla[i] > max) {
13         max = cisla[i];
14     }
15 }
16
```

cs

### Pole jako parametr funkce

#### C++ – pole jako parametr

Pole se v C++ předává jako **ukazatel** na první prvek. Velikost pole se obvykle předává jako samostatný parametr.

cpp

```
1 // Pole jako parametr – předává se ukazatel
2 void vypisPole(int pole[], int velikost) {
3     for (int i = 0; i < velikost; i++) {
4         cout << pole[i] << " ";
5     }
6     cout << endl;
7 }
8
9 // Alternativní zápis s ukazatelem
10 void vypisPole(int* pole, int velikost) {
11     for (int i = 0; i < velikost; i++) {
12         cout << pole[i] << " ";
13     }
14     cout << endl;
15 }
16
17 // Změna hodnot v poli
18 void nasobDvema(int pole[], int velikost) {
19     for (int i = 0; i < velikost; i++) {
20         pole[i] *= 2; // změni původní pole
21     }
22 }
23
24 // Konstantní pole (nelze měnit)
25 void vypisKonstantniPole(const int pole[], int velikost) {
26     for (int i = 0; i < velikost; i++) {
27         cout << pole[i] << " ";
28         // pole[i] = 10; // CHYBA – nelze měnit
29     }
30 }
31
32 // Použití
33 int main() {
34     int cisla[] = {1, 2, 3, 4, 5};
35     int velikost = sizeof(cisla) / sizeof(cisla[0]);
36
37     vypisPole(cisla, velikost);
38     nasobDvema(cisla, velikost);
39     vypisPole(cisla, velikost); // {2, 4, 6, 8, 10}
40
41     return 0;
42 }
```

#### Vrácení pole z funkce:

cpp

```
1 // Dynamicky alokované pole
2 int* vytvorPole(int velikost) {
3     int* pole = new int[velikost];
4     for (int i = 0; i < velikost; i++) {
5         pole[i] = i + 1;
6     }
7     return pole;
8 }
9
10 int main() {
11     int* pole = vytvorPole(5);
12     // použití pole...
13     delete[] pole; // nutné uvolnit paměť!
14     return 0;
15 }
```

#### C# – pole jako parametr

V C# se pole předává jako **reference**, což znamená, že změny v metodě ovlivní původní pole.

cs

```
1 // Pole jako parametr
2 static void VypisPole(int[] pole) {
3     foreach (int cislo in pole) {
4         Console.WriteLine(cislo + " ");
5     }
6     Console.WriteLine();
7 }
8
9 // Změna hodnot v poli
10 static void NasobDvema(int[] pole) {
11     for (int i = 0; i < pole.Length; i++) {
12         pole[i] *= 2; // změni původní pole
13     }
14 }
15
16 // Pole s params – proměnný počet argumentů
17 static int Soucet(params int[] cisla) {
18     int vysledek = 0;
19     foreach (int cislo in cisla) {
20         vysledek += cislo;
21     }
22     return vysledek;
23 }
24
25 // Vrácení pole z metody
26 static int[] VytvorPole(int velikost) {
27     int[] pole = new int[velikost];
28     for (int i = 0; i < velikost; i++) {
29         pole[i] = i + 1;
30     }
31     return pole;
32 }
33
34 // Použití
35 static void Main() {
36     int[] cisla = {1, 2, 3, 4, 5};
37
38     VypisPole(cisla); // 1 2 3 4 5
39     NasobDvema(cisla);
40     VypisPole(cisla); // 2 4 6 8 10
41
42     // params – různé způsoby volání
43     Console.WriteLine(Soucet(1, 2, 3)); // 6
44     Console.WriteLine(Soucet(10, 20, 30, 40)); // 100
45
46     // Vrácení pole
47     int[] novePole = VytvorPole(5);
48     VypisPole(novePole); // 1 2 3 4 5
49 }
```

### Statické metody třídy Array (C#)

Třída **System.Array** poskytuje statické metody pro práci s poli.

#### Řazení pole

cs

```
1 int[] cisla = {5, 2, 8, 1, 9};
2
3 // Seřazení vzestupně
4 Array.Sort(cisla); // {1, 2, 5, 8, 9}
5
6 // Obrácení pořadí
7 Array.Reverse(cisla); // {9, 8, 5, 2, 1}
8
9 // Řazení stringů
10 string[] jmena = {"Karel", "Anna", "Petr", "Eva"};
11 Array.Sort(jmena); // {Anna, Eva, Karel, Petr}
```

#### Vyhledávání v poli

cs

```
1 int[] cisla = {1, 2, 5, 8, 9};
2
3 // Binární vyhledávání (pole musí být seřazené!)
4 int index = Array.BinarySearch(cisla, 5); // 2
5
6 if (index >= 0) {
7     Console.WriteLine($"Nalezeno na indexu {index}");
8 } else {
9     Console.WriteLine("Nenalezeno");
10 }
11
12 // IndexOf – najde první výskyt
13 int[] cisla2 = {10, 20, 30, 20, 40};
14 int pozice = Array.IndexOf(cisla2, 20); // 1
15
16 // LastIndexOf – najde poslední výskyt
17 int posledni = Array.LastIndexOf(cisla2, 20); // 3
```

#### Kopírování pole

cs

```
1 int[] puvodni = {1, 2, 3, 4, 5};
2
3 // Kopírování celého pole
4 int[] kopie = new int[5];
5 Array.Copy(puvodni, kopie, 5);
6
7 // Kopírování části pole
8 int[] cast = new int[3];
9 Array.Copy(puvodni, 1, cast, 0, 3); // {2, 3, 4}
10
11 // Clone – vytvoří mělkou kopii
12 int[] klon = (int[])puvodni.Clone();
```

#### Vyplnění pole

cs

```
1 int[] cisla = new int[5];
2
3 // Vyplnění celého pole jednou hodnotou
4 Array.Fill(cisla, 10); // {10, 10, 10, 10, 10}
5
6 // Vyplnění části pole
7 Array.Fill(cisla, 5, 1, 3); // {10, 5, 5, 5, 10}
8 //          hodnota, start, count
```

#### Další užitečné metody

cs

```
1 int[] cisla = {1, 2, 3, 4, 5};
2
3 // Vymazání pole (nastavení na výchozí hodnoty)
4 Array.Clear(cisla, 0, cisla.Length); // {0, 0, 0, 0, 0}
5
6 // Změna velikosti (vytvoří nové pole)
7 Array.Resize(ref cisla, 10); // rozšíří na 10 prvků
8
9 // Existence prvku
10 bool existuje = Array.Exists(cisla, x => x > 3); // true/false
11
12 // Najdi prvek dle podmínky
13 int[] cisla2 = {1, 2, 3, 4, 5};
14 int prvni = Array.Find(cisla2, x => x > 3); // 4
15 int[] vsechny = Array.FindAll(cisla2, x => x > 2); // {3, 4, 5}
16
17 // ForEach – provede akci pro každý prvek
18 Array.ForEach(cisla2, x => Console.WriteLine(x + " "));
```

#### Instanční metody pole (C#)

Každé pole v C# má vlastnosti a metody zděděné od **System.Array**.

#### Vlastnosti pole

cs

```
1 int[] cisla = {10, 20, 30, 40, 50};
2
3 // GetLength – délka v dané dimenzi
4 int pocet = cisla.Length; // 5
5
6 // Rank – počet dimenzí (1 pro jednorozměrné pole)
7 int dimeenze = cisla.Rank; // 1
8
9 // IsReadOnly – zda je pole pouze pro čtení
10 bool readOnly = cisla.IsReadOnly; // false
11
12 // IsFixedSize – zda má pole pevnou velikost
13 bool fixed = cisla.IsFixedSize; // true
```

#### Instance metody

cs

```
1 int[] cisla = {1, 2, 3, 4, 5};
2
3 // GetLength – délka v dané dimenzi
4 int delka = cisla.GetLength(0); // 5
5
6 // GetValue – získání hodnoty na indexu (jako objekt)
7 object hodnota = cisla.GetValue(2); // 3
8
9 // SetValue – nastavení hodnoty na indexu
10 cisla.SetValue(100, 2); // cisla[2] = 100
11
12 // Clone – vytvoří kopii pole
13 int[] kopie = (int[])cisla.Clone();
14
15 // CopyTo – zkopíruje do jiného pole
16 int[] cil = new int[10];
17 cisla.CopyTo(cil, 2); // zkopíruje od indexu 2
```

#### LINQ metody pro pole

Pole v C# podporují LINQ dotazy:

cs

```
1 using System.Linq;
2
3 int[] cisla = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
4
5 // Where – filtrování
6 int[] suda = cisla.Where(x => x % 2 == 0).ToArray(); // {2, 4, 6, 8, 10}
7
8 // Select – transformace
9 int[] druhe = cisla.Select(x => x * x).ToArray(); // {1, 4, 9, 16, ...}
10
11 // Sum – součet
12 int soucet = cisla.Sum(); // 55
13
14 // Average – průměr
15 double prumer = cisla.Average(); // 5.5
16
17 // Min / Max
18 int min = cisla.Min(); // 1
19 int max = cisla.Max(); // 10
20
21 // Count – počet prvků splňujících podmínku
22 int pocetVetsichNezPet = cisla.Count(x => x > 5); // 5
23
24 // Any – existuje alespoň jeden prvek splňující podmínku
25 bool existujeSude = cisla.Any(x => x % 2 == 0); // true
26
27 // All – všechny prvky splňují podmínku
28 bool vsechnaKladna = cisla.All(x => x > 0); // true
29
30 // First / Last
31 int prvni = cisla.First(); // 1
32 int posledni = cisla.Last(); // 10
33
34 // FirstOrDefault – vrátí první nebo výchozí hodnotu
35 int prvniVetsiNezDeset = cisla.FirstOrDefault(x => x > 10); // 0
36
37 // OrderBy / OrderByDescending
38 int[] serazene = cisla.OrderByDescending(x => x).ToArray(); // {10, 9, 8, ...}
39
40 // Take / Skip
41 int[] prvnichPet = cisla.Take(5).ToArray(); // {1, 2, 3, 4, 5}
42 int[] bezPrvnichPeti = cisla.Skip(5).ToArray(); // {6, 7, 8, 9, 10}
43
44 // Distinct – unikátní hodnoty
45 int[] sOpakovanim = {1, 2, 2, 3, 3, 3, 4};
46 int[] unikatni = sOpakovanim.Distinct().ToArray(); // {1, 2, 3, 4}
```

#### Příklady praktického použití

##### Statistiky pole:

cs

```
1 int[] znamky = {1, 2, 1, 3, 2, 1, 4, 3, 2};
2
3 int pocet = znamky.Length; // 9
4 double prumer = znamky.Average(); // 2.11
5 int nejlepsi = znamky.Min(); // 1
6 int nejhorsi = znamky.Max(); // 4
7 int pocetJednicek = znamky.Count(z => z == 1); // 3
8
9 Console.WriteLine($"Počet známek: {pocet}");
10 Console.WriteLine($"Průměr: {prumer:F2}");
11 Console.WriteLine($"Nejlepší: {nejlepsi}");
12 Console.WriteLine($"Nejhorší: {nejhorsi}");
13 Console.WriteLine($"Počet jedniček: {pocetJednicek}");
```

##### Filtrace a transformace:

cs

```
1 string[] jmena = {"Anna", "Petr", "Karel", "Eva", "Martin"};
2
3 // Jména začínající na 'A' nebo 'E', převedená na velká písmena
4 var vysledek = jmena
5     .Where(j => j.StartsWith("A") || j.StartsWith("E"))
6     .Select(j => j.ToUpper())
7     .ToArray();
8
9 // výsledek: {"ANNA", "EVA"}
10
11 // Seřazená jména delší než 4 znaky
12 var dlouha = jmena
13     .Where(j => j.Length > 4)
14     .OrderBy(j => j)
15     .ToArray();
16
17 // dlouha: {"Karel", "Martin", "Petr"}
```